

Kubernetes Storage — Zero to Hero

This guide explains Kubernetes storage from absolute basics to advanced production-level architectures. It covers PersistentVolumes (PV), PersistentVolumeClaims (PVC), StorageClasses, StatefulSets, Deployments, EBS vs EFS, AccessModes (RWO/ROX/RWX), dynamic provisioning, replication patterns, storage internals, node-level mounting, and real-world production edge cases.

Kubernetes Storage: Complete Zero-to-Hero Guide

Table of Contents

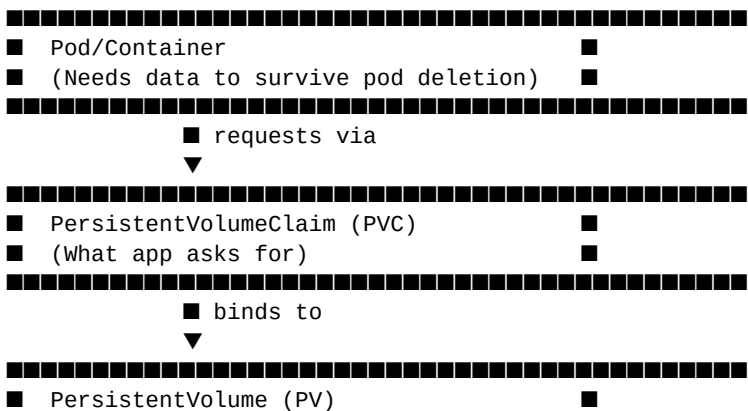
- 1. Storage Fundamentals
- 2. PersistentVolumes (PV)
- 3. PersistentVolumeClaims (PVC)
- 4. StorageClasses
- 5. Access Modes & Reclaim Policies
- 6. StatefulSets vs Deployments
- 7. Replicas & Storage Strategies
- 8. Advanced Scenarios
- 9. Best Practices

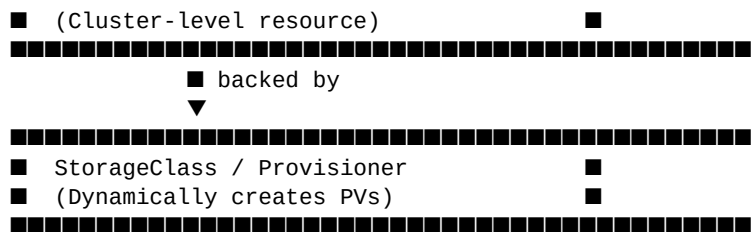
Part 1: Storage Fundamentals

The Problem

By default, Kubernetes containers have ephemeral storage. When a pod crashes or is deleted, data is lost. We need persistent storage that survives pod restarts.

The Solution: Three-Layer Architecture





Key Concepts

Concept	Purpose	Scope
StorageClass	Define storage types & provisioning rules	Cluster-wide
PersistentVolume (PV)	Cluster resource representing actual storage	Cluster-wide
PersistentVolumeClaim (PVC)	Request for storage by pod	Namespace-scoped
Volume	Mount point inside pod	Pod-scoped

Part 2: PersistentVolumes (PV) - Deep Dive

What is a PV?

A PV is a cluster-level resource that represents an actual piece of storage. It exists independent of any pod.

Creating a PV Manually (Static Provisioning)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-local-storage
spec:
  capacity:
    storage: 10Gi # Size of storage
  accessModes:
    - ReadWriteOnce # How many nodes can access
  persistentVolumeReclaimPolicy: Delete # What happens after PVC deletion
  storageClassName: manual # Associated StorageClass

# One of these (storage backend)
hostPath:
  path: /mnt/data

# OR for cloud providers
```

```
# awsElasticBlockStore:
#   volumeID: vol-12345
# OR
# gcePersistentDisk:
#   pdName: gce-disk-1
```

PV Attributes Explained

1. **Capacity**

```
capacity:
  storage: 10Gi    # Requests <= this amount
```

- The storage size available
- PVCs can request less than this
- Kubernetes doesn't enforce maximum (containers can fill it)

2. **Access Modes**

Controls how many nodes can mount the volume:

```
accessModes:
-ReadWriteOnce    # RW0 - Only 1 node, read & write
-ReadOnlyMany     # ROX - Multiple nodes, read-only
-ReadWriteMany    # RWX - Multiple nodes, read & write
```

Mode	Single Node	Multiple Nodes	Read	Write
RWO	Yes	No	✓	✓
ROX	Yes	Yes	✓	✗
RWX	Yes	Yes	✓	✓

Real-world:

- Local/Block storage (AWS EBS, GCP Persistent Disk): RWO only
- Network storage (NFS, GlusterFS): RWX
- Shared config: ROX

3. **Reclaim Policy**

What happens to the PV after PVC deletion:

```
persistentVolumeReclaimPolicy: Delete # Default for dynamic
```

Policy	Behavior	Use Case
--------	----------	----------

Delete	PV & data deleted	Non-critical data, auto-cleanup
Retain	PV kept, manual cleanup	Important data, recover later
Recycle	Data wiped, PV reused	Deprecated, use Delete

■ ■ ****CRITICAL****: Delete policy removes data. Retain requires manual cleanup.

4. ****Storage Backend Options****

```
# Local node storage
hostPath:
  path: /mnt/data
  type: DirectoryOrCreate # Directory, DirectoryOrCreate, File

# Cloud providers
awsElasticBlockStore:
  volumeID: vol-12345
  fsType: ext4

gcePersistentDisk:
  pdName: gce-disk-1
  fsType: ext4

azureDisk:
  diskName: disk1
  diskURI: /subscriptions/.../disk1

# Network storage
nfs:
  server: 192.168.1.1
  path: /exports/data

# Other
iscsi:
  targetPortal: 192.168.1.1:3260
  iqn: iqn.2016-04.example.com:storage.disk1
```

5. ****Node Affinity**** (Advanced)

Restrict which nodes can mount the PV:

```
nodeAffinity:
  required:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
              - node-1
              - node-2
```

PV Lifecycle

Available → Bound → Released → Available
(PVC binds) (PVC deleted)

Part 3: PersistentVolumeClaims (PVC)

What is a PVC?

A PVC is how pods request storage. It's like a purchase order for storage capacity.

Creating a PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-claim
  namespace: default          # Namespaced resource
spec:
  storageClassName: standard  # Which StorageClass to use
  accessModes:
    - ReadWriteOnce          # Must match or be subset of PV
  resources:
    requests:
      storage: 5Gi           # How much to request
    limits:
      storage: 10Gi          # Max allowed (optional)
  selector:
    matchLabels:
      type: fast             # Match specific PV by label
```

PVC Attributes

1. **Storage Amount**

```
resources:
  requests:
    storage: 5Gi          # Must be <= PV capacity
```

2. **Access Modes**

```
accessModes:
  - ReadWriteOnce        # Must match PV's supported modes
```

3. **StorageClass**

```
storageClassName: standard    # Name of StorageClass
# If empty string "", no provisioning (manual PV binding)
```

```
# If omitted, uses default StorageClass
```

4. **Label Selector**

```
selector:
  matchLabels:
    type: fast      # PV must have these labels
```

Using PVC in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: nginx:latest
      volumeMounts:
        - name: data      # Name of volume (below)
          mountPath: /data # Where to mount in container
          readOnly: false
        - name: config
          mountPath: /etc/config
          readOnly: true

  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: my-claim # PVC name

    - name: config
      persistentVolumeClaim:
        claimName: config-claim
        readOnly: true      # Force read-only in pod
```

PVC States

```
Pending → Bound → Lost
(waiting (PV found) (PV deleted,
for PV)      data at risk)
```

Part 4: StorageClass - Dynamic Provisioning

The Problem with Manual PVs

- Manual PV creation doesn't scale

- Requires cluster admin involvement
- Wastes time in development

The Solution: StorageClass

Automatically creates PVs on-demand using provisioners.

Creating a StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com          # What creates PVs
allowVolumeExpansion: true           # Can PVC size grow?
volumeBindingMode: WaitForFirstConsumer # When to bind PV
reclaimPolicy: Delete                # Delete PV after PVC deletion

# Provisioner-specific parameters
parameters:
  type: gp3                          # AWS EBS type
  iops: "3000"                       # IO operations/second
  throughput: "125"                  # MB/s
  encrypted: "true"
  deleteOnTermination: "true"

# For GCP
#provisioner: kubernetes.io/gce-pd
#parameters:
#  type: pd-ssd
#  replication-type: regional-pd

# For Azure
#provisioner: disk.csi.azure.com
#parameters:
#  storageaccounttype: Standard_LRS
#  kind: Managed
```

StorageClass Attributes

1. **Provisioner**

Determines what plugin creates the PV:

```
provisioner: ebs.csi.aws.com
# Built-in:
# - kubernetes.io/aws-ebs
# - kubernetes.io/gce-pd
# - kubernetes.io/azure-disk
# CSI (modern):
# - ebs.csi.aws.com
```



```
# - pd.csi.storage.gke.io
# - disk.csi.azure.com
# - csi.rbd.ceph.com
# Custom:
# - your.company.com/storage
```

2. **Volume Binding Mode**

When to bind PV to PVC:

```
volumeBindingMode: Immediate      # Bind immediately (default)
volumeBindingMode: WaitForFirstConsumer # Wait for pod to schedule
```

Immediate: Good for most cases, but:

- PV provisioned even if pod can't schedule
- Wastes resources in multi-zone clusters

WaitForFirstConsumer: Better for replicas:

- Pod scheduled first → PV created in same zone
- Avoids cross-zone data transfer costs
- Required for StatefulSets with multiple replicas

3. **Allow Volume Expansion**

```
allowVolumeExpansion: true    # PVC size can increase
```

Usage:

```
kubectl patch pvc my-claim -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'
```

Warning: Not all provisioners support expansion (check CSI driver).

4. **Reclaim Policy**

```
reclaimPolicy: Delete      # Delete PV after PVC deletion
reclaimPolicy: Retain      # Keep PV, manual cleanup needed
```

5. **Parameters** (Provisioner-Specific)

AWS:

```
parameters:
  type: gp3                # gp2 (general), io1 (provisioned), st1 (throughput)
  iops: "3000"             # For gp3/io1: IOPS
  throughput: "125"        # For gp3: MB/s
  encrypted: "true"
  kmsKeyId: "arn:aws:..."
```

GCP:

```
parameters:
  type: pd-ssd          # pd-standard or pd-ssd
  replication-type: regional-pd
```

StorageClass Examples by Cloud

****AWS (EBS)****

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-gp3-fast
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  type: gp3
  iops: "5000"
  throughput: "250"
  encrypted: "true"
reclaimPolicy: Delete
```

****GCP (Persistent Disk)****

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gcp-ssd-regional
provisioner: pd.csi.storage.gke.io
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: pd-ssd
  replication-type: regional-pd
allowVolumeExpansion: true
reclaimPolicy: Delete
```

****NFS (Network File System)****

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-slow
provisioner: example.com/nfs-provisioner
parameters:
  server: "192.168.1.10"
  path: "/exports"
  archiveOnDelete: "true"
allowVolumeExpansion: true
reclaimPolicy: Retain
```

Setting Default StorageClass

```
kubectl patch storageclass fast-ssd -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}
```

Access Modes Matrix

Choosing Access Mode

- Use: Databases, Single-replica apps
- Example: MySQL, PostgreSQL, MongoDB

****ReadOnlyMany (ROX)****

- Use: Shared config, Read-heavy distributed apps
- Example: Config files, ML models, shared libraries

****ReadWriteMany (RWX)****

- Use: Shared data between multiple pods
- Example: Collaborative apps, shared uploads, cache

```
accessModes:
  - ReadWriteMany
```

Reclaim Policy Decisions

****Delete (Automatic Cleanup)****

```
reclaimPolicy: Delete
```

✓ Good for: Dev/Test, temporary data, cost control

✗ Bad for: Production data, disaster recovery

Example:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: temp-storage
provisioner: ebs.csi.aws.com
reclaimPolicy: Delete
```

****Retain (Manual Cleanup)****

```
reclaimPolicy: Retain
```

✓ Good for: Production databases, important data

✗ Bad for: Wasteful, creates orphaned PVs

Cleanup manually:

```
# Remove PVC
kubectl delete pvc my-claim

# PV still exists (Released state)
kubectl get pv

# Delete PV manually
kubectl delete pv pv-name

# Finally, delete the actual storage (AWS/GCP/Azure console)
```

Part 6: StatefulSets vs Deployments with Storage

The Fundamental Difference

****Deployments****

- Pods are identical and interchangeable
- No guaranteed pod names

- Random pod IDs
- No stable network identity
- Not suitable for stateful apps

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: app
          image: nginx:latest
          volumeMounts:
            - name: data
              mountPath: /data
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: shared-claim # All pods use SAME PVC

```

■ ■ ****Problem****: All replicas share one PVC, leading to:

- Write conflicts
- Lock contention
- Poor performance
- Data corruption risk

****StatefulSets****

- Pods have stable, ordered identities
- Guaranteed pod names: `name-0`, `name-1`, `name-2`
- Each pod gets its own PVC
- Stable network identity (headless service)
- Designed for stateful apps

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: database

```

```

spec:
  serviceName: database      # REQUIRED: headless service
  replicas: 3
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql

volumeClaimTemplates:      # Auto-creates PVC per replica
- metadata:
  name: data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "standard"
    resources:
      requests:
        storage: 10Gi

```

✓ Each replica gets unique PVC: `data-database-0`, `data-database-1`, etc.

StatefulSet vs Deployment Comparison

Feature	Deployment	StatefulSet
Pod Identity	Random	Stable (name-0, name-1)
Network Identity	Ephemeral	Stable DNS
Storage	Shared PVC	One PVC per pod
Scale Down	Instant, random	Ordered (remove highest #)
Scale Up	Instant	Sequential, ordered
Use Case	Stateless web	Databases, queues
Complexity	Simple	Moderate

When to Use StatefulSet

****Perfect fit:****

- Databases (MySQL, PostgreSQL, MongoDB)

- Message queues (RabbitMQ, Kafka)
- Search engines (Elasticsearch)
- Cache (Redis)
- Any app requiring persistent unique identity

```
# Example: Kafka StatefulSet
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
spec:
  serviceName: kafka
  replicas: 3
  selector:
    matchLabels:
      app: kafka
  template:
    metadata:
      labels:
        app: kafka
    spec:
      containers:
        - name: kafka
          image: confluentinc/cp-kafka:7.0.0
          env:
            - name: KAFKA_BROKER_ID
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: KAFKA_ADVERTISED_LISTENERS
              value: "PLAINTEXT://$(POD_NAME).kafka:9092"
          volumeMounts:
            - name: data
              mountPath: /var/lib/kafka

      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ "ReadWriteOnce" ]
            storageClassName: "fast-ssd"
            resources:
              requests:
                storage: 100Gi
```

Required: Headless Service for StatefulSet

```
apiVersion: v1
kind: Service
metadata:
  name: kafka
spec:
  clusterIP: None          # REQUIRED: headless service
  selector:
    app: kafka
```

```
ports:
- port: 9092
  name: broker
- port: 9093
  name: controller
```

Benefits:

- Pods accessible via DNS: `kafka-0.kafka.default.svc.cluster.local`
- Pod 0 always at same address
- Network identity independent of IP

Part 7: Replicas & Storage Strategies

Strategy 1: Shared Storage (Anti-pattern for Databases)

```
# ■ BAD: All replicas share one PVC
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: app
        volumeMounts:
        - name: data
          mountPath: /data
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: shared-pvc # One PVC, three pods
```

****Problems:****

- Write conflicts
- No isolation
- Hard to debug
- Poor performance

****When it's OK:****

- Read-only data (config, models)
- True stateless apps
- Distributed lock implementation

Strategy 2: Per-Replica Storage (Correct for Stateful)

```
# ✓ GOOD: Each replica has own PVC
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:14
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data

volumeClaimTemplates:
- metadata:
  name: data
  spec:
    storageClassName: "ebs-gp3"
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 50Gi
```

PVCs created:

- `data-postgres-0` (on node-1)
- `data-postgres-1` (on node-2)
- `data-postgres-2` (on node-3)

****Advantages:****

- Complete isolation

- Parallel processing
- Failure independence
- Replication via app logic (PostgreSQL streaming replication)

Strategy 3: Hybrid - Shared + Per-Replica

```
# Some shared, some per-replica
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: app
spec:
  serviceName: app
  replicas: 3
  template:
    spec:
      containers:
        - name: app
          volumeMounts:
            # Per-replica local cache
            - name: local-cache
              mountPath: /cache
            # Shared read-only config
            - name: config
              mountPath: /etc/config
              readOnly: true

      volumes:
        - name: config
          persistentVolumeClaim:
            claimName: shared-config-pvc
            readOnly: true

      volumeClaimTemplates:
        - metadata:
            name: local-cache
          spec:
            accessModes: [ "ReadWriteOnce" ]
            storageClassName: "fast-ssd"
            resources:
              requests:
                storage: 10Gi
```

Strategy 4: Init Container + Shared Storage

Copy data on startup:

```

apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  initContainers:
  - name: init-data
    image: busybox
    command: ['sh', '-c', 'cp -r /template/* /data/']
    volumeMounts:
    - name: template
      mountPath: /template
      readOnly: true
    - name: data
      mountPath: /data

  containers:
  - name: app
    image: myapp:latest
    volumeMounts:
    - name: data
      mountPath: /data

  volumes:
  - name: template
    persistentVolumeClaim:
      claimName: template-pvc
      readOnly: true
  - name: data
    persistentVolumeClaim:
      claimName: data-pvc

```

Replication Architectures

1. Master-Slave (Active-Passive)

```

Master (replica-0) ----writes----> Slave (replica-1, replica-2)
  ↓                               ↑
  ■■■■■■■■ replication log sync ■■■■■■■■→

```

Example: MySQL with one PVC master, slaves read replicated data

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  template:
    spec:
      initContainers:
      - name: init-mysql
        image: mysql:8.0
        command:

```

```

- bash
- -c
- |
  ordinal=$(hostname | rev | cut -d- -f1 | rev)
  if [ "$ordinal" -eq 0 ]; then
    # Master setup
    mysql_install_db
  else
    # Slave setup
    sleep 30 # Wait for master
  fi
volumeMounts:
- name: data
  mountPath: /var/lib/mysql

containers:
- name: mysql
  image: mysql:8.0
  env:
  - name: SERVER_ID
    valueFrom:
      fieldRef:
        fieldPath: metadata.name
  volumeMounts:
  - name: data
    mountPath: /var/lib/mysql

volumeClaimTemplates:
- metadata:
  name: data
  spec:
    storageClassName: "standard"
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 100Gi

```

****2. Peer-to-Peer (Active-Active)****

```

Replica-0 ↔ Replica-1 ↔ Replica-2
(data)      (data)      (data)

```

Example: Elasticsearch cluster

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: elasticsearch
spec:
  serviceName: elasticsearch
  replicas: 3
  template:
    spec:
      containers:
      - name: elasticsearch
        image: docker.elastic.co/elasticsearch/elasticsearch:8.0.0
        env:
        - name: discovery.seed_hosts
          value: "elasticsearch-0,elasticsearch-1,elasticsearch-2"
        - name: cluster.initial_master_nodes

```

```

        value: "elasticsearch-0,elasticsearch-1,elasticsearch-2"
    volumeMounts:
      - name: data
        mountPath: /usr/share/elasticsearch/data

    volumeClaimTemplates:
      - metadata:
          name: data
        spec:
          storageClassName: "standard"
          accessModes: [ "ReadWriteOnce" ]
          resources:
            requests:
              storage: 100Gi

```

Part 8: Advanced Scenarios

Scenario 1: Multiple Storage Classes by Purpose

```

# High-performance SSD for databases
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "5000"
  throughput: "250"
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer

# Standard HDD for archives
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard-hdd
provisioner: ebs.csi.aws.com
parameters:
  type: st1
  throughput: "125"
allowVolumeExpansion: true

# NFS for shared data
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: shared-nfs
provisioner: nfs.csi.k8s.io
parameters:

```

```
server: "192.168.1.10"
path: "/exports"
allowVolumeExpansion: true
reclaimPolicy: Retain
```

Scenario 2: StatefulSet with Init Container for Data Migration

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: migrated-db
spec:
  serviceName: migrated-db
  replicas: 3
  template:
    spec:
      initContainers:
        # Download data from S3
        - name: migrate-data
          image: amazon/aws-cli:latest
          command:
            - sh
            - -c
            - |
              aws s3 cp s3://my-bucket/db-backup.tar.gz /data/
              cd /data && tar -xzf db-backup.tar.gz
      volumeMounts:
        - name: data
          mountPath: /data

      containers:
        - name: postgres
          image: postgres:14
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data

  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        storageClassName: "standard"
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 100Gi
```

Scenario 3: Expanding PVC Size

When data grows, expand PVC dynamically:

```
# Edit the PVC
kubectl edit pvc postgres-data-0

# Change storage size
spec:
```

```

resources:
  requests:
    storage: 100Gi    # Was 50Gi

# Kubernetes expands the volume
# Pod restarts may be needed depending on storage class

```

Scenario 4: Pod-to-Pod Communication with Data Sharing

```

# Producer StatefulSet
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: producer
spec:
  serviceName: producer
  replicas: 2
  template:
    spec:
      containers:
        - name: producer
          image: myapp:producer
          volumeMounts:
            - name: shared
              mountPath: /data

  volumeClaimTemplates:
    - metadata:
        name: shared
      spec:
        storageClassName: nfs-shared
        accessModes: [ "ReadWriteMany" ] # Shared access
        resources:
          requests:
            storage: 50Gi

---
# Consumer Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: consumer
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: consumer
          image: myapp:consumer
          volumeMounts:
            - name: shared
              mountPath: /data
              readOnly: true

      volumes:
        - name: shared
          persistentVolumeClaim:
            claimName: shared-data-pvc
            readOnly: true

```

Scenario 5: Local Storage for Performance

For ultra-high-performance requirements (trade-off: single node only):

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: cache
spec:
  serviceName: cache
  replicas: 3
  template:
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - cache
              topologyKey: kubernetes.io/hostname

      containers:
        - name: redis
          image: redis:7
          volumeMounts:
            - name: data
              mountPath: /data

  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        storageClassName: local-ssd
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 100Gi

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-ssd
provisioner: kubernetes.io/local
volumeBindingMode: WaitForFirstConsumer
```

Scenario 6: Snapshot & Restore for Backups

```
# Create a snapshot of a PVC
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
```



```

    name: postgres-backup-2024
spec:
  volumeSnapshotClassName: csi-snapshotter
  source:
    persistentVolumeClaimName: data-postgres-0

---
# Restore from snapshot
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: restored-data
spec:
  storageClassName: standard
  accessModes:
    - ReadWriteOnce
  dataSource:
    name: postgres-backup-2024
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
resources:
  requests:
    storage: 50Gi

---

```

Part 9: Best Practices & Decision Tree

Decision Tree: Which Storage Solution?

```

Need persistent data?
■ NO → Use emptyDir (pod-scoped, ephemeral)
■
■ YES
  ■ Need multiple pods to access simultaneously?
    ■ NO (Database, Queue, etc.)
      ■ Use StatefulSet + volumeClaimTemplates
      ■ One PVC per replica
    ■ YES (Shared config, cache)
      ■ Read-only?
        ■ YES → ROX (ReadOnlyMany)
      ■ Read-Write shared?
        ■ YES → RWX (ReadWriteMany) + NFS/GlusterFS
        ■ Use Deployment + single shared PVC

```

Storage Solution by Use Case

Use Case	Solution	StorageClass	Access Mode	Replicas
MySQL/PostgreSQL	StatefulSet + volumeClaimTemplates	standard	RWO	PerReplica

MongoDB Replica Set	StatefulSet + volumeClaimTemplates	fast-ssd	RWO	PerReplica
Redis Cluster	StatefulSet + volumeClaimTemplates	fast-ssd	RWO	PerReplica
Elasticsearch	StatefulSet + volumeClaimTemplates	fast-ssd	RWO	PerReplica
Kafka	StatefulSet + volumeClaimTemplates	fast-ssd	RWO	PerReplica
Shared Logs	Deployment + PVC	nfs	RWX	Shared
Config Files	ConfigMap (no storage)	-	-	-
ML Models	Deployment + PVC	standard	ROX	ROX
User Uploads	Deployment + PVC	standard	RWX	Shared
Cache	StatefulSet + local	local-ssd	RWO	PerReplica

Performance Optimization

****For highest throughput:****

```
storageClassName: fast-ssd
volumeBindingMode: WaitForFirstConsumer # Avoid cross-zone
parameters:
  type: gp3
  iops: "5000"
  throughput: "250"
```

****For cost savings:****

```
storageClassName: standard-hdd
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: st1
  throughput: "125"
```

****For development:****

```
storageClassName: local
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
```

Production Checklist

- ☐ Use StatefulSet for databases/stateful apps
- ☐ Each replica has own PVC (volumeClaimTemplates)
- ☐ StorageClass has `reclaimPolicy: Retain` for important data

- [] `volumeBindingMode: WaitForFirstConsumer` to avoid cross-zone costs
- [] `allowVolumeExpansion: true` for flexibility
- [] Regular backups via VolumeSnapshot
- [] Monitor storage usage: `kubectl get pvc`
- [] Test disaster recovery procedures
- [] Document which StorageClass for each app
- [] Set resource requests for PVC sizing
- [] Use headless Service for StatefulSet
- [] Configure pod affinity to spread replicas

Troubleshooting

****PVC stuck in Pending:****

```
kubectl describe pvc my-claim
# Check events for provisioner errors
# Verify StorageClass exists
kubectl get storageclass
```

****Pod can't mount PVC:****

```
kubectl describe pod pod-name
# Check: accessModes, node affinity, PV status
kubectl get pv
```

****Performance issues:****

```
# Check which node PVC is on
kubectl get pvc -o wide

# Check I/O metrics
kubectl top pod

# Consider using local storage for latency-sensitive apps
```

Quick Reference: YAML Templates

StatefulSet Template

```
apiVersion: apps/v1
```

```

kind: StatefulSet
metadata:
  name: app
spec:
  serviceName: app
  replicas: 3
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - app
              topologyKey: kubernetes.io/hostname

      containers:
        - name: app
          image: myimage:latest
          ports:
            - containerPort: 5432
              name: db
          volumeMounts:
            - name: data
              mountPath: /data
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"

      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            storageClassName: standard
            accessModes: [ "ReadWriteOnce" ]
            resources:
              requests:
                storage: 10Gi

---
apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  clusterIP: None
  ports:
    - port: 5432
      targetPort: 5432

```

```
name: db
selector:
  app: app
```

StorageClass Template (AWS)

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: ebs.csi.aws.com
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
```

Summary

Concept	Key Points
PV	Cluster resource, actual storage, created manually or automatically
PVC	Namespace resource, request for storage, pod uses via volume
StorageClass	Defines provisioner & parameters, enables dynamic PV creation
StatefulSet	For stateful apps, each pod gets unique PVC, stable identity
Deployment	For stateless apps, shared storage if needed
RWO	Single node read-write, typical for databases
RWX	Multiple node read-write, needs NFS/distributed storage
Reclaim	Delete = auto-remove, Retain = manual cleanup

****The golden rule****: StatefulSet + volumeClaimTemplates + WaitForFirstConsumer for all stateful applications.